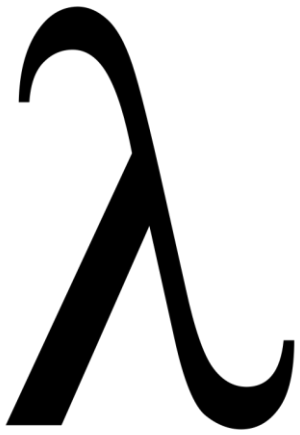


ML Lab 1

Programmazione Funzionale

2025/2026

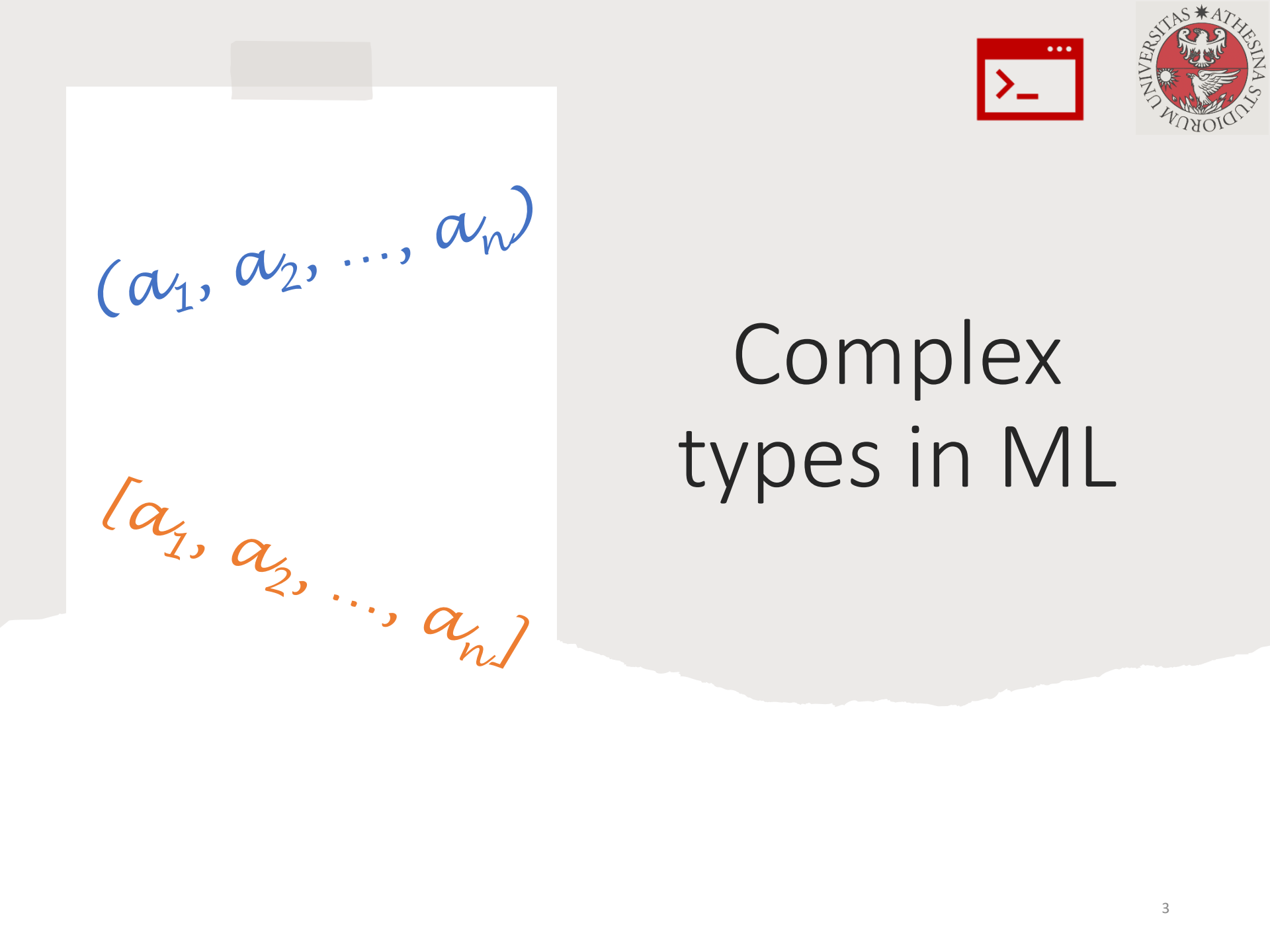
Università di Trento

Chiara Di Francescomarino

Tutoring

- Starting from next Tuesday
  - 17:30 – 18:30 in PC A202
  - Except 17-03-2026 17:30 – 18:30 in PC A201

- If you need, from next week, you can reserve a slot on Friday morning (10:30 – 11:30) in Moodle



Complex types in ML

$(a_1, a_2, \ldots, a_n)$

Tuples in ML

Tuples

- We have already seen something when looking at the operators
- Example

```
> val t = (1, 2, 3);
val t = (1, 2, 3): int * int * int
```

- We can mix types in tuple definitions

```
> val t = (4, 5.0, "six");
val t = (4, 5.0, "six"): int * real * string
```

- We can also use a complex type instead of a simple one

```
> val t = (1, (2, 3, 4));
val t = (1, (2, 3, 4)): int * (int * int * int)
```

Accessing tuple components:

- Let us consider

```
> val t = (4, 5.0, "six");
```

- Then

```
> #1 (t);
val it = 4: int
> #3 (t);
val it = "six": string
> #4 (t);
poly: : error: Type error in function application.
Function: #4 : 'a * 'b * 'c * 'd -> 'd
Argument: (t) : int * real * string
Reason: Can't unify {4: 'a} to int * real * string (Field 4
missing)
```

Generic type: the specific type is determined only when we use it.

$[a_1, a_2, \ldots, a_n]$

Lists in ML

Lists

- Represented with square brackets
- Example

```
> [1,2,3];
val it = [1, 2, 3]: int list
```

- Note that all elements of a list (unlike tuples) must be of the same type

More examples

```
> [1.0,2.0];
val it = [1.0, 2.0]: real list

> [1,2.0];
poly: : error: Elements in a list have different types.
Item 1: 1 : int
Item 2: 2.0 : real
Reason:
Can't unify int (*In Basis*) with real (*In Basis*)
(Different type constructors)

> [];
val it = []: 'a list
```

Generic type: the specific type is determined only when we use it.

Note that for an empty list, ML cannot determine the type of the elements

Head of a list: `hd`

- Head: first element of a list

```
> val L = [2,3,4];
val L = [2, 3, 4]: int list
> val M = [5];
val M = [5]: int list

> hd(L);
val it = 2: int
> hd(M);
val it = 5: int
```

Tail of a list: `tl`

- Tail: all the rest

```
> L;
val it = [2, 3, 4]: int list
> M;
val it = [5]: int list

> tl (L);
val it = [3, 4]: int list
> tl (M);
val it = []: int list
```

Note that ML can determine the type of this empty list

Concatenation of lists: @

- Example

```
> [1,2] @ [3,4];
val it = [1, 2, 3, 4]: int list
```

- Note that both lists must be of the same type

```
> [1,2] @ ["a","b"];
poly: : error: Type error in function application.
Function: @ : int list * int list -> int list
Argument: ([1, 2], ["a", "b"]) : int list * string list
Reason:
Can't unify int (*In Basis*) with string (*In Basis*)
(Different type constructors)
```

- Two different types of concatenation
  - ^: Strings
  - @: List

Cons: ::

- An operator that takes an element of type `'a` and a list of type `'a list` and combines them

```
> 2 :: [3,4];
val it = [2, 3, 4]: int list
> 2 :: nil;
val it = [2]: int list
> 2 :: [];
val it = [2]: int list
```

`nil` is the same as the empty list `[]`

- :: is right associative

```
> 1 :: 2 :: 3 :: nil;
val it = [1, 2, 3]: int list
> (1 :: (2:: (3::nil)));
val it = [1, 2, 3]: int list
```

- The other way would make no sense

Strings to lists: `explode`

```
> explode ("abcd");
val it = [#"a", #"b", #"c", #"d"]: char list
> explode ("");
val it = []: char list
```

Lists to strings: `implode`

```
> implode ([ #"a", #"b", #"c", #"d"]);
val it = "abcd": string
> implode (nil);
val it = "": string
> implode (explode ("xyz"));
val it = "xyz": string
```

The ML type system

- Basic types `int`, `real`, `bool`, `char`, `string`, `unit`
- Complex types: For now, 2 constructors:
  - `T1 * T2 * ... * Tn (tuples)`
  - `T list`

Examples

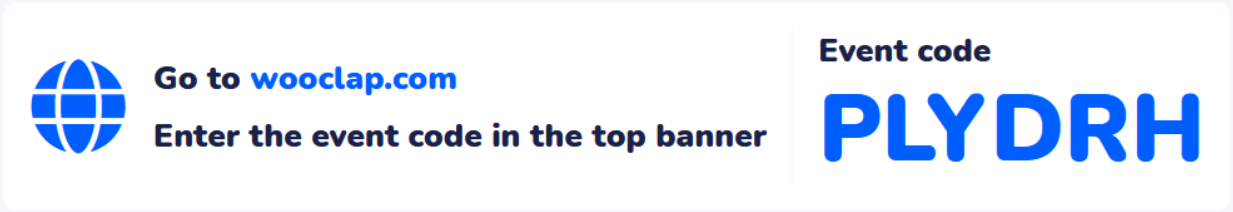
```
> [1, 2, 3];
val it = [1, 2, 3]: int list

> ("ab", [1,2,3], 4);
val it = ("ab", [1, 2, 3], 4): string * int
list * int

> [[(1,2),(3,4)], [(5,6)], nil];
val it = [[(1, 2), (3, 4)], [(5, 6)], []]:
(int * int) list list
```

A list of a int*int list

What is the error in the following expression (without trying it ☺)?

```
> #"a"^#"b";

poly: : error: Value or constructor (^#) has not
been declared Found near #"a" ^# "b"

poly: : error: Type error in function
application.

   Function: #"a" : char

   Argument: ^# : bad

   Reason: Value being applied does not have a
function type

Found near #"a" ^# "b"

Static Errors
```

Some questions … without trying it ☺



Exercise 1.1

- Write the expression to convert
  - 123.45 to the next lower integer
  - -123.45 to the next lower integer
  - 123.45 to the next higher integer
  - -123.45 to the next higher integer

Exercise 1.2

- Write the expression to convert
  - #”Y” to an integer
  - 120 to a character
  - 97.0 to a character
  - #”N” to a real
  - #”Z” to a string

Exercise 1.3

- How can we fix the errors in the following expressions?
  - `ceil(4);`
  - `if true then 5+6 else 7.0;`

Exercise 1.4

- How can we fix the errors in the following expressions?
  - `if 0 then 1 else 2;`
  - `ord("a")`

Few more questions

Go to wooclap.com

Enter the event code in the top banner

Event code

PLYDRH

Copy participation link

Waiting for participants ...

Exercise 1.5

- Fix the following expressions

```
explode ["bar"];

implode ( #"a", #"b") ;

["r"]::["a","t"];
```

Exercise 1.6

- Give examples of objects of the following types, without using empty lists

```
int list list list

(int * char) list

string list * ( int * (real * string))
* int
```

Exercise 1.7

- Give examples of objects of the following types, without using empty lists

```
((int * int) * (bool list) * real) *
(real * string)

(bool * int) * char

real * int list list list list
```